

# AI活用(5) ソフトウェア開発(1)

コード生成

情報科学専攻 | 松野 裕

2026年7月1日 (対面)

# 本日のアジェンダ

---

1. AIとソフトウェア開発の現在（10分）
2. AIによるコード生成の基礎（20分）
3. 主要ツールの紹介と使い方（15分）
4. コード生成の実践テクニック（10分）
5. 演習（45分）

# AIとソフトウェア開発の現在

AI搭載開発ツールの急速な普及

# AI搭載開発ツールの普及

---

## 主要なツール

- **GitHub Copilot** — コード補完・生成（学生無料）
- **Cursor** — AI統合IDE
- **Google Gemini Pro** — コード生成・レビュー（学生無料）
- **ChatGPT / Claude** — 対話的なコード開発

開発者の92%がAIコーディングツールを利用（GitHub 2024年調査）

AIは「コードを書く」作業を大きく変えつつある

# AI活用で変わる開発スタイル

|   | 従来の開発     | AI活用の開発        |
|---|-----------|----------------|
| 1 | 仕様を理解     | 仕様を理解          |
| 2 | 設計を考える    | 設計を考える         |
| 3 | コードを一から書く | AIにコード生成を依頼    |
| 4 | デバッグ      | AIにデバッグ支援を依頼   |
| 5 | テスト作成     | AIにテスト生成を依頼    |
| 6 | ドキュメント作成  | AIにドキュメント生成を依頼 |

人間の役割は「書く」から「指示する・レビューする」へ

# AIによるコード生成の基礎

基本フローと効果的なプロンプト

# コード生成の基本フロー

1. 自然言語で仕様を説明
- ↓
2. AIがコードを生成
- ↓
3. 人間がレビュー・修正
- ↓
4. テスト・動作確認
- ↓
5. 必要に応じて再度AIに改善を依頼

## 重要なポイント

- AIが生成したコードを **そのまま使わない**
- 必ず **理解してからレビュー** する
- **動作確認とテスト** は人間の責任

# 効果的なコード生成プロンプト

---

## 4つのポイント

1. 言語・フレームワークを明示する
2. 入出力の仕様を具体的に書く
3. 制約条件を伝える
4. コードスタイルを指定する



# 良いプロンプトの例

---

Python3で、CSVファイルを読み込み、指定した列の平均値と標準偏差を計算する関数を書いてください。

- pandasを使用
- エラー処理を含める（ファイル未存在、列名不正）
- 型ヒントを付ける
- docstringを含める

# Bad prompt vs Good prompt

|       | Bad prompt      | Good prompt                                            |
|-------|-----------------|--------------------------------------------------------|
| 内容    | CSVを読み込むコードを書いて | Python3で以下の関数を書いてください。関数名:<br><code>analyze_csv</code> |
| 入出力   | 不明              | 入力: ファイルパス(str) , 列名(str) / 出力: dict(mean, std, count) |
| ライブラリ | 不明              | pandas使用                                               |
| エラー処理 | 指示なし            | FileNotFoundError, ValueError                          |
| スタイル  | 指示なし            | 型ヒント・docstring付き                                       |

具体的であるほど、期待通りのコードが生成される

# 主要ツールの紹介と使い方

Gemini Pro / GitHub Copilot / ChatGPT・Claude

# Gemini Proでのコード生成

---

## 特徴

- 学生は **無料** で利用可能
- 長いコンテキストを扱える
- Google Colabとの連携が便利

## 実践例

Python3で、matplotlibを使って以下のデータを棒グラフにするコードを書いてください。 データ：  
{ 'A': 85, 'B': 92, 'C': 78, 'D': 95 } 要件： タイトル付き、Y軸ラベル付き、色はグラデーション

生成されたコードはGoogle Colabですぐに実行・確認できる

# GitHub Copilot（学生無料）

---

## セットアップ

1. [GitHub Education](#) に学生認証を申請
2. VS Codeに **GitHub Copilot拡張機能** をインストール
3. GitHubアカウントでサインイン

## 使い方

- **コメントを書く** → コード補完が表示される
- `Tab` キーで補完を確定
- `Ctrl+Enter` で複数の候補を表示

```
# CSVファイルを読み込み、各列の統計量を表示する関数  
# ← このコメントを書くだけでCopilotがコードを提案
```

# ChatGPT / Claude でのコード生成

---

## 対話的な開発が可能

ユーザー: 「データを読み込む関数を書いて」 → AI: コードを生成  
ユーザー: 「エラー処理を追加して」 → AI: 修正版を生成  
ユーザー: 「テストも書いて」 → AI: テストコードを生成

## メリット

- **対話的に修正** できる
- 前のコンテキストを覚えている
- 説明を求めることもできる

# コード生成の実践テクニック

段階的開発・テスト駆動・リファクタリング・デバッグ

# テクニック1: 段階的な開発

一度に全部作らず、関数単位で生成・検証

## 進め方

Step 1: データ読み込み関数を生成 → 動作確認  
Step 2: データ処理関数を生成 → 動作確認  
Step 3: 可視化関数を生成 → 動作確認  
Step 4: メイン処理を組み立て → 全体テスト

## なぜ段階的がよいか

- エラーの原因を 特定しやすい
- 各関数の品質を確保できる
- AIも短いコードの方が正確に生成する



# テクニック2: テスト駆動

---

先にテストケースを書かせ、それを満たすコードを生成

Step 1: 「この仕様に対するpytestのテストを書いてください」  
Step 2: テストを実行（当然失敗する）  
Step 3: 「このテストを通るコードを書いてください」  
Step 4: テストを実行して確認

## メリット

- 仕様が **明確** になる
- 品質が **保証** される
- リグレッション（退行）を防げる

## テクニック3: リファクタリング依頼

以下のコードをより効率的にリファクタリングしてください。 - 可読性を向上させる - 重複コードを削除する - 適切な関数に分割する - 変更箇所と理由を説明してください

### よくあるリファクタリング

- 長い関数を分割
- マジックナンバーを定数に
- ネストを減らす
- 命名を改善

# テクニック4: デバッグ支援

---

## エラーメッセージを貼り付けて原因と修正を聞く

以下のPythonコードを実行すると、このエラーが出ます。原因と修正方法を教えてください。 [コード] ... [エラーメッセージ] `TypeError: 'NoneType' object is not subscriptable`

## ポイント

- エラーメッセージ **全文** を貼る
- 関連するコードも含める
- 何をしようとしていたか説明する

# 演習

データ処理・コードレビュー・デバッグ

# AI演習 演習1（20分）：データ処理スクリプト作成

---

## Gemini Proでデータ処理スクリプトを作成

課題: 以下の機能を持つPythonスクリプトを段階的に作成

1. CSVファイルの読み込み（サンプルデータを使用）
2. 基本統計量の計算（平均、中央値、標準偏差）
3. 簡単なグラフの作成（棒グラフまたは折れ線グラフ）

## 進め方

- 段階的な開発 を実践（関数ごとに生成）
- Google Colabで実行・確認
- サンプルCSVデータは配布

## AI演習 演習2（15分）：生成コードのレビューと修正

---

### AIが生成したコードの問題点を見つける

1. 演習1で生成したコードを読み返す
2. 以下の観点でチェック：
  - **エラー処理** は十分か？
  - **エッジケース** は考慮されているか？
  - **命名** は適切か？
  - **効率性** に問題はないか？
3. AIにレビューを依頼して比較

## AI演習 演習3（10分）：AIにデバッグさせる

---

### エラーを意図的に入れ、AIにデバッグさせる

1. 演習1のコードに 意図的にバグを入れる
  - 例: 変数名のタイポ、インデックスエラー、型エラー
2. エラーメッセージとコードをAIに渡す
3. AIの修正提案を確認

### 考察

- AIはどの程度正確にバグを見つけられるか？
- AIが見逃すバグはどんなものか？

# AI生成コードの注意点

---

## セキュリティリスク

- AIが生成するコードに脆弱性が含まれる場合がある
- 入力値のバリデーションが不十分なことがある

## ライセンス問題

- 学習データに含まれるコードのライセンスに注意
- OSSライセンスに違反していないか確認



# AI生成コードの注意点（続き）

---

## テストの重要性

- AI生成コードも **必ずテスト** する
- 「動く」と「正しい」は違う
- 特にエッジケースは要注意

AIの出力を 鵜呑みにせず、必ず検証する 習慣を持つ

# 課題

---

## AIを使ったPythonスクリプト作成

テーマ: 自分の研究に関連するデータ処理

### 提出物

1. Pythonスクリプト (.py または .ipynb)
2. 作成過程の記録 (使用プロンプト、修正内容と理由)
3. 実行結果のスクリーンショット

提出期限: 次回授業 (7/8) まで

# 次回予告

---

## 第12回: AI活用(6) ソフトウェア開発(2) — テストと品質

- AIによるテストコード生成
- AIによるコードレビュー
- リファクタリングとドキュメント生成

### 準備

- 今回の課題で作成したコードを持参
- 次回はそのコードにテストを追加します